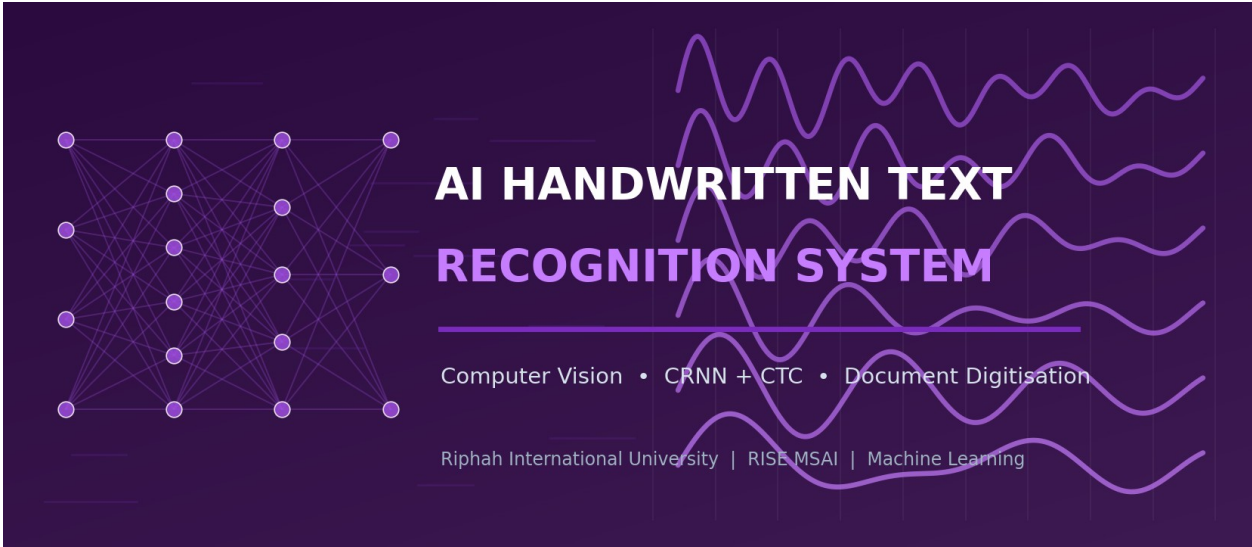




FINAL PROJECT PROPOSAL

AI-Based Handwritten Text
Recognition System

A Final-Year Machine-Learning Project Proposal

Riphah International University

Department of RISE MSAI

Course / Subject: Machine Learning

Submitted By — Group Members	Submitted To
1. Azhar Hussain (Group Lead) 2. Rahma 3. Haris Javaid 4. Syed Farakh Abbas	Junaid Khan Course Instructor — Machine Learning

Date of Submission: 11 May 2026

Academic Session 2025–2026 | Confidential — For Academic Evaluation Only

Table of Contents

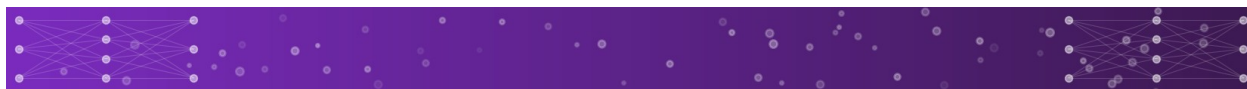
If the entries below do not display page numbers in your viewer, place the cursor in the table and press Ctrl+A then F9 (Word) — “Update Field → Update entire table”.

2. Abstract.....	4
3. Introduction.....	4
4. Problem Statement.....	5
5. Objectives.....	5
6. Scope of the Project.....	6
In Scope.....	6
Out of Scope (Limitations).....	6
7. Literature Review.....	6
7.1 Key Works.....	6
7.2 Comparison of HTR Approaches.....	7
7.3 Datasets & Resources.....	7
8. Proposed Methodology.....	8
8.1 System Architecture.....	8
8.2 Step-by-Step Workflow.....	8
8.3 Workflow Flowchart.....	9
8.4 Algorithms, Models & Frameworks.....	10
9. Technologies and Tools.....	10
10. Dataset Information.....	11
10.1 Candidate Datasets.....	11
10.2 Data Pre-processing.....	11
10.3 Training / Validation / Testing Split.....	11
10.4 Data Augmentation.....	11
11. Expected Outcomes.....	12
12. Innovation / Uniqueness.....	12
13. Feasibility Analysis.....	13
13.1 Technical Feasibility.....	13
13.2 Cost Analysis.....	13
13.3 Hardware Feasibility & GPU / Resource Limitations.....	13
13.4 Practical Implementation Plan.....	13
14. Hardware and Software Requirements.....	14
15. Future Enhancements.....	14
16. Timeline / Gantt Chart.....	14
17. Conclusion.....	16

18. References.....16

2. Abstract

Vast amounts of human knowledge — lecture notes, examination scripts, historical manuscripts, medical records, official registers, filled forms — still exist only as handwriting on paper. Converting this content into editable, searchable digital text unlocks indexing, translation, accessibility and long-term preservation. This project proposes an AI-Based Handwritten Text Recognition (HTR) system that takes an image, scanned page or PDF of handwritten text and outputs editable text, exportable as TXT, DOCX or a searchable PDF. The recognition engine is a Convolutional Recurrent Neural Network (CRNN): a CNN feature extractor (a compact 7-layer stack or a MobileNet-style backbone) feeds a bidirectional LSTM sequence encoder, which is decoded with Connectionist Temporal Classification (CTC) — the architecture introduced by Shi et al. for image-based sequence recognition. Around this core the system adds image pre-processing (deskewing, binarisation, line and word segmentation), a dictionary/edit-distance spell-correction post-processor (SymSpell), basic layout reconstruction, and an optional second model for Urdu/regional scripts. The system is trained on the IAM Handwriting Database (and synthetic data generated from handwriting fonts) using a free Google Colab GPU. Targeted performance is a word-level Character Error Rate (CER) of roughly 8–12 % on IAM lines without a language model and lower with post-processing — comfortably achievable by a student team — together with a usable desktop/web application for everyday note digitisation.



Keywords: *Handwritten text recognition, OCR, CRNN, CTC loss, BiLSTM, transfer learning, document digitisation, IAM database, multi-language OCR.*

3. Introduction

Optical Character Recognition (OCR) for printed text is largely a solved problem, but handwritten text recognition remains challenging because handwriting is highly variable — slant, spacing, letter shapes, ligatures and pen pressure differ between writers and even within one writer's page. Classical HTR pipelines first segmented a line into individual characters and classified each one, a brittle approach that fails when letters touch or overlap. The breakthrough came with segmentation-free sequence learning: an image of an entire word or text line is processed by a CNN to extract a feature sequence, a recurrent network models the left-to-right context, and a CTC layer aligns the variable-length prediction to the variable-length label without needing per-character segmentation. Shi, Bai and Yao's CRNN (2015) established this end-to-end design and remains the practical workhorse of HTR; later work (Puigcerver 2017; Bluche & Messina 2017) refined the convolutional and recurrent blocks, and recent Transformer models such as TrOCR (2021) push accuracy further at much higher computational cost.

This project deliberately builds on the CRNN+CTC design because it is accurate, well documented, and trainable on a free cloud GPU. To make the system genuinely useful rather than a bare model, we add a complete document pipeline: pre-processing (deskew, binarise, denoise), line/word segmentation, the CRNN recogniser, a spell-

correction post-processor, simple layout reconstruction, and export to editable formats. Multi-language support (an English model plus an optional Urdu/Arabic-script model) and a clean user interface complete the deliverable.

4. Problem Statement

Organisations and individuals accumulate large volumes of handwritten material that cannot be searched, edited, translated or efficiently archived. Manual transcription is the current fallback, but it is:

- **Costly:** Extremely time-consuming and expensive at scale (thousands of pages of notes, forms, registers).
- **Inaccurate:** Error-prone and inconsistent, especially for unfamiliar handwriting or domain jargon.
- **Inaccessible:** A barrier to accessibility — handwritten notes are not screen-reader friendly for visually impaired users.
- **Fragile:** An obstacle to preservation — paper degrades, and without digitisation the content is at risk.

Formally: given an input image / scan / PDF page P containing handwritten text, produce an editable digital transcript T that closely matches the written content (low character and word error rates), preserving basic layout (lines, paragraphs), supporting more than one language/script, and exporting to TXT / DOCX / searchable PDF — all runnable on student hardware. Research models address the core recognition step; this project additionally delivers the surrounding, user-facing document workflow.

5. Objectives

- **Primary objective:** Implement a CRNN + CTC handwriting recogniser achieving a line-level CER of roughly 8–12 % on the IAM test set without a language model (lower with post-processing).
- **Secondary objective:** Build a robust pre-processing stage: orientation correction, binarisation, denoising, and line/word segmentation for multi-line documents.
- **Secondary objective:** Add a dictionary + edit-distance spell-correction post-processor (SymSpell) to reduce word-level errors.
- **Secondary objective:** Provide multi-language support — an English model plus an optional Urdu/Arabic-script model — with a simple language selector.
- **Secondary objective:** Reconstruct basic layout (line order, paragraphs) and export results as TXT, DOCX and searchable PDF.
- **Secondary objective:** Deliver a clean desktop/web UI for uploading images/PDFs and reviewing/editing the recognised text, including a near-real-time webcam capture mode.
- **Learning objective:** Demonstrate competence in CNNs, RNN/LSTM sequence modelling, CTC loss, transfer learning, data augmentation and OCR evaluation (CER/WER).

6. Scope of the Project

In Scope

- Recognition of handwritten Latin-script (English) text from images, scans and PDF pages.
- Pre-processing: deskew, binarise, denoise, and line/word segmentation for multi-line documents.
- CRNN+CTC recognition with greedy and beam-search decoding; SymSpell post-correction.
- Optional second model for Urdu/Arabic-script words (proof-of-concept, smaller dataset).
- Basic layout reconstruction and export to TXT, DOCX and searchable PDF.
- A desktop/web application with file upload, webcam capture and an editable results view.

Out of Scope (Limitations)

- Full historical-manuscript transcription with complex multi-column layouts, marginalia and tables.
- Cursive recognition in arbitrary languages beyond the trained models; mathematical-formula recognition.
- Guaranteed accuracy on extremely messy, faded or low-resolution handwriting outside the training distribution.
- Large-scale Transformer models (e.g. full TrOCR fine-tuning) — noted as a future enhancement only.
- Production-grade hosting, user accounts and document-management features.

7. Literature Review

Handwritten text recognition has progressed from per-character segmentation, through HMM- and MDLSTM-based line recognisers, to the now-standard CNN+RNN+CTC family and, most recently, Transformer-based encoder-decoder models. The works below frame our design choices.

7.1 Key Works

Graves et al. (2009) showed that multi-dimensional LSTMs with a CTC output layer could recognise unconstrained handwriting at the line level without explicit segmentation, winning ICDAR competitions. Shi, Bai and Yao (2015) introduced the CRNN — convolutional layers for feature extraction, bidirectional LSTMs for sequence modelling, and a CTC transcription layer — as a unified, end-to-end trainable network for image-based sequence recognition; this is the backbone of our system. Puigcerver (2017) demonstrated that carefully designed 1-D (rather than 2-D) recurrent layers on convolutional features are sufficient and faster, setting strong IAM baselines, while Bluche and Messina (2017) proposed gated CNN-BLSTM architectures for multilingual HTR. Graves (2012) and Graves et al. (2006) provide the theoretical basis of CTC. More recently, TrOCR (Li et al., 2021) couples a pretrained vision Transformer encoder with a pretrained text Transformer decoder, achieving very low error rates but requiring substantial compute and large pretraining corpora — beyond our scope, though an obvious future direction. Open-source engines such as Tesseract 4/5 (LSTM-based) and toolkits like PyLaia, Kraken and PaddleOCR offer reusable components and

baselines. SymSpell (Garbe) supplies fast symmetric-delete spelling correction for the post-processing stage.

7.2 Comparison of HTR Approaches

Approach (Year)	Core Idea	Strength	Limitation / Cost	IAM CER (approx.)
Char-segmentation + CNN	Segment line→chars, classify each	Simple components	Brittle; touching letters break it	high / unstable
MDLSTM + CTC (2009)	2-D LSTM over the image + CTC	Segmentation-free; historic SOTA	Slow; hard to train	≈ 18 %
CRNN (CNN+BLSTM+CTC) (2015) ★	CNN features → BiLSTM → CTC	Accurate, end-to-end, light, well-documented	Needs decent data & augmentation	≈ 10-12 %
Puigcerver CNN-1D-LSTM (2017)	Conv features + 1-D LSTMs	Faster; strong IAM baseline	Still needs careful tuning	≈ 8 %
Gated CNN-BLSTM (2017)	Gated conv + BLSTM, multilingual	Good cross-language results	More complex architecture	≈ 8 %
TrOCR (2021)	ViT encoder + Transformer decoder	Very low error; no CTC needed	Large model; heavy pretraining/compute	≈ 3-4 %
Tesseract 4/5 (LSTM)	Open-source LSTM line recogniser	Free, ready-to-use, many languages	Weaker on free-form handwriting	varies

Table 2.1: Representative HTR approaches. ★ = backbone adopted (CRNN + CTC, with a lightweight CNN front-end). CER figures are indicative line-level results reported in the literature on the IAM database.

7.3 Datasets & Resources

Resource	Content	Use Here
IAM Handwriting Database	≈ 1 539 scanned forms, 13 353 labelled lines, 657 writers (English)	Primary training/validation/testing data
IAM-OnDB / CVL / RIMES	Additional handwriting corpora (online strokes / multi-writer / French)	Optional extra data and robustness checks
Synthetic handwriting (TRDG / fonts)	Text rendered with many handwriting fonts + distortions	Pre-training and augmentation to enlarge the training set
UrduDoc / UNHD / KHATT (Arabic-script)	Urdu/Arabic handwritten text datasets	Optional second-language proof-of-concept
SymSpell dictionary, NLTK/WordFreq	Word lists and frequencies	Spell-correction post-processing

Table 2.2: Datasets and resources considered.

Research gap addressed: most academic HTR work optimises the recogniser in isolation on IAM lines. Far fewer student projects deliver the full document workflow — pre-processing, segmentation, post-correction, layout rebuild, multi-language, and

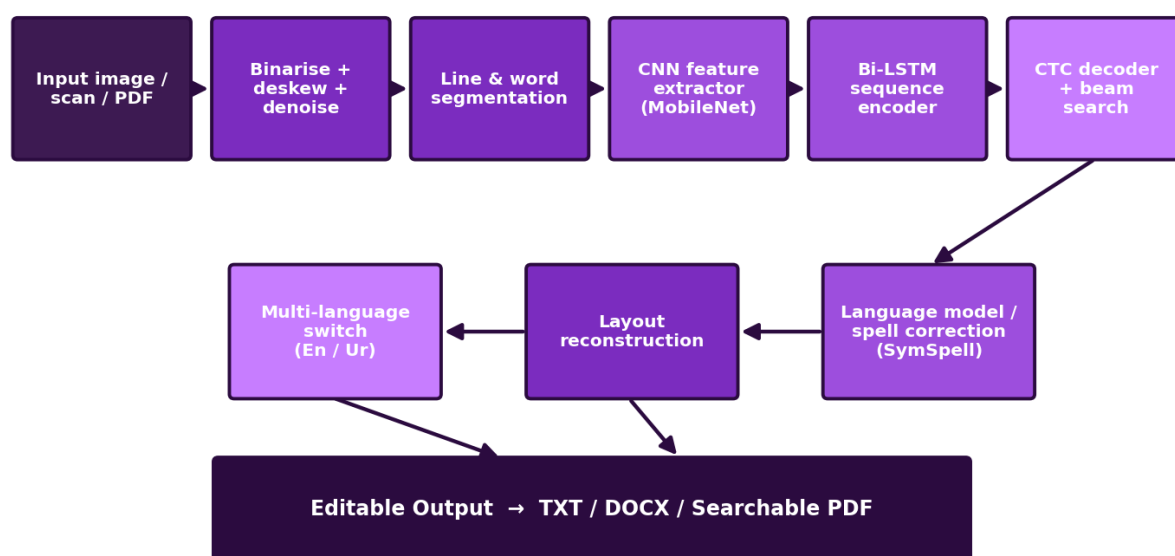
export — wrapped in a usable app. This project targets that practical, end-to-end gap on free hardware.

8. Proposed Methodology

The methodology has three stages: a document pre-processing/segmentation stage, a CRNN+CTC recognition stage (trained once, served many times), and a post-processing/export/UI stage.

8.1 System Architecture

System Architecture — Handwritten Text Recognition



Pipeline stages flow left → right (top) then converge into the application layer (bottom).

Figure 2.1: End-to-end architecture — image/scan/PDF → pre-processing & segmentation → CRNN (CNN + BiLSTM + CTC) → spell-correction & layout rebuild → editable TXT/DOCX/searchable-PDF output.

8.2 Step-by-Step Workflow

- Input handling** — Accept JPG/PNG images, scanned pages, multi-page PDFs (rasterised), or near-real-time webcam frames.
- Pre-processing** — Convert to grayscale; correct skew (Hough/projection-profile); binarise (Otsu / adaptive / Sauvola); remove noise and ruled lines; normalise contrast.
- Segmentation** — Detect text regions, then segment into lines (horizontal projection / connected components) and, where needed, into words; normalise each line image height (e.g. 32–64 px).
- Recogniser construction** — Build the CRNN: a CNN feature extractor (compact 7-conv stack or MobileNet-style backbone, optionally ImageNet-initialised) → map-to-sequence → 2 stacked bidirectional LSTM layers → linear layer over the character set → CTC.
- Training** — Train end-to-end with CTC loss on IAM (plus synthetic data) using Adam, learning-rate scheduling and gradient clipping on a free Colab/Kaggle GPU; checkpoint regularly; monitor validation CER/WER.

6. **Decoding** — Use greedy (best-path) decoding for speed and CTC beam search (optionally with a small n-gram language model) for accuracy.
7. **Post-processing** — Apply SymSpell dictionary correction; restore casing/punctuation heuristics; reassemble words → lines → paragraphs in reading order.
8. **Multi-language** — Provide a language selector that swaps in an alternative model (e.g. an Urdu/Arabic-script CRNN) and an appropriate dictionary.
9. **Export & UI** — Render the result in an editable text panel beside the source image; export to TXT, DOCX (python-docx) and searchable PDF (OCR text layer over the image).
10. **Testing & evaluation** — Measure CER/WER on held-out IAM data and on a small custom test set of team members' handwriting; analyse error types; document results.

8.3 Workflow Flowchart

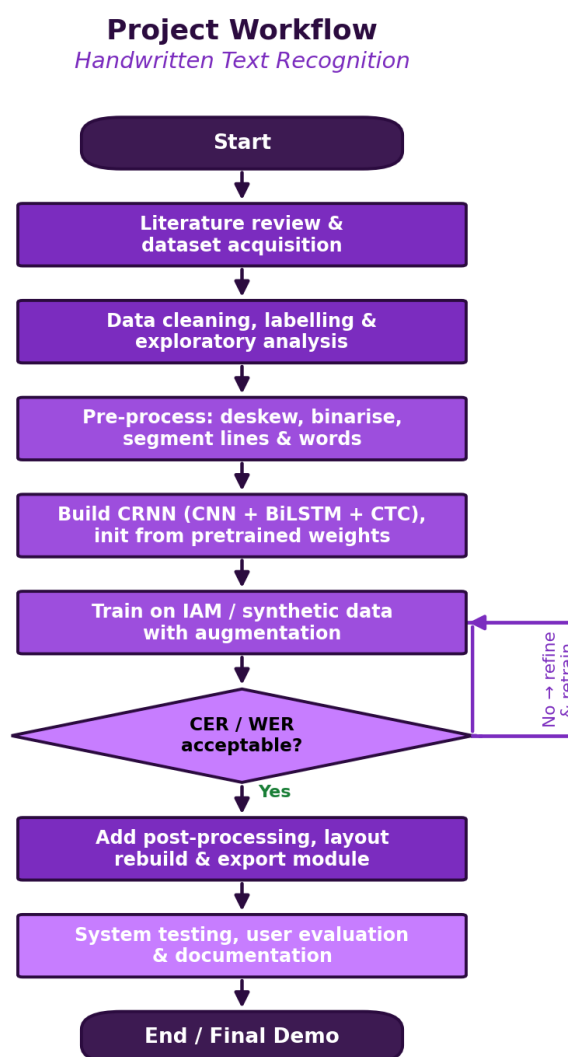


Figure 2.2: Project workflow with the model-quality decision gate and refinement loop.

8.4 Algorithms, Models & Frameworks

- **CRNN (primary):** CNN feature extractor + 2× BiLSTM + CTC transcription layer.
- **CTC loss / decoders:** Connectionist Temporal Classification training loss; greedy and beam-search decoding.
- **CNN backbone options:** Compact 7-layer conv stack (VGG-like) or MobileNetV3-small features (transfer learning).
- **Pre-processing algorithms:** Otsu/Sauvola binarisation, projection-profile deskew & line segmentation, connected components.
- **Post-processing:** SymSpell symmetric-delete spelling correction; optional KenLM n-gram language model in beam search.
- **Baselines:** Tesseract 5 (LSTM) and a per-character CNN classifier for comparison.
- **Frameworks:** PyTorch / TensorFlow-Keras, OpenCV, NumPy, scikit-image, python-docx, pdf2image/PyMuPDF, Streamlit/PyQt.

9. Technologies and Tools

Category	Tools / Technologies	Purpose in the Project
Programming language	Python 3.10+	Data prep, modelling, post-processing, application
Deep-learning framework	PyTorch (alt. TensorFlow / Keras)	Building and training the CRNN; CTC loss
Computer vision / imaging	OpenCV, scikit-image, Pillow	Binarisation, deskew, denoise, line/word segmentation
Numerical	NumPy	Array operations, feature handling
Pretrained backbones	torchvision MobileNetV3 / ResNet18 (ImageNet)	Optional transfer-learning CNN front-end
OCR baselines / tools	Tesseract 5, (ref.) PaddleOCR / PyLaia	Comparison baselines and reusable components
Post-processing / NLP	SymSpell, NLTK / wordfreq, (opt.) KenLM	Spell correction and language modelling
Document I/O & export	pdf2image / PyMuPDF, python-docx, reportlab	Read PDFs; export TXT/DOCX/searchable PDF
App / UI	Streamlit (alt. PyQt / Flask)	Upload, webcam capture, editable results view, export buttons
Training environment	Google Colab / Kaggle Notebooks (free GPU)	GPU training without local high-end hardware
Dev tools	VS Code, Jupyter, Git/GitHub	Coding, experiments, version control, collaboration
Dataset sources	IAM database; synthetic (TRDG); (opt.) Urdu/Arabic sets	Training, validation, testing, multi-language PoC

Table: Technology stack selected for cost-free, low-resource development.

10. Dataset Information

10.1 Candidate Datasets

- **IAM Handwriting Database:** $\approx 1\,539$ scanned forms $\rightarrow 13\,353$ labelled text lines from 657 writers (English, modern Latin script); the standard HTR benchmark.
- **Synthetic handwriting data:** Text strings rendered with dozens of handwriting fonts and random distortions (rotation, elastic warp, noise) using tools like TextRecognitionDataGenerator; used for pre-training and to enlarge the training set.
- **Optional extra corpora:** IAM-OnDB, CVL, RIMES for additional robustness; KHATT/UNHD/UrduDoc for an optional Urdu/Arabic-script proof-of-concept.
- **Custom test set:** A small set of scanned notes/forms in the team members' own handwriting for a realistic end-to-end evaluation.

10.2 Data Pre-processing

- Extract line images and ground-truth transcripts from the IAM XML; filter unreadable or wrongly-segmented samples; build a fixed character vocabulary (letters, digits, punctuation, space, CTC blank).
- Normalise each line image: grayscale, deskew, binarise (or keep grayscale), pad/resize to a fixed height (e.g. 32–64 px) while preserving aspect ratio; right-pad widths within a batch.
- Tokenise transcripts to integer label sequences; record sequence lengths for CTC.
- For multi-line documents at inference time: deskew the page, segment into lines via projection profiles/connected components, then feed each line to the model.

10.3 Training / Validation / Testing Split

Split	Proportion	Purpose
Training	$\approx 70\text{--}75\%$	Learn CRNN weights (IAM official train set + synthetic data)
Validation	$\approx 10\text{--}15\%$	Hyper-parameter tuning, early stopping, decoder/LM selection
Testing	$\approx 15\%$	Final unbiased CER/WER (IAM official test set + custom handwriting set)

Table 2.3: Data partitioning. The IAM writer-independent split is respected so that no writer appears in both training and testing.

10.4 Data Augmentation

Augmentation is critical because handwriting datasets are limited and writer-dependent: random rotations ($\pm 3\text{--}5^\circ$), slant/shear, elastic and grid distortions, random scaling and translation, morphological erosion/dilation (stroke thinning/thickening), brightness/contrast jitter, Gaussian/salt-pepper noise, and random background textures. Large volumes of synthetic handwriting can be generated to pre-train the network before fine-tuning on real IAM data.

11. Expected Outcomes

- A trained CRNN+CTC recogniser with line-level CER $\approx$ 8–12 % on IAM without a language model, improving with beam search + SymSpell post-correction.
- A working document pipeline: deskew $\rightarrow$ binarise $\rightarrow$ line/word segmentation $\rightarrow$ recognition $\rightarrow$ post-correction $\rightarrow$ layout reconstruction.
- Multi-language capability: an English model plus an optional Urdu/Arabic-script proof-of-concept model.
- Export to TXT, DOCX and searchable PDF, preserving basic line/paragraph structure.
- A desktop/web application with file/PDF upload, near-real-time webcam capture, and an editable side-by-side review panel.
- Complete documentation: code repository, training notebooks, evaluation report (CER/WER, error analysis) and user guide.

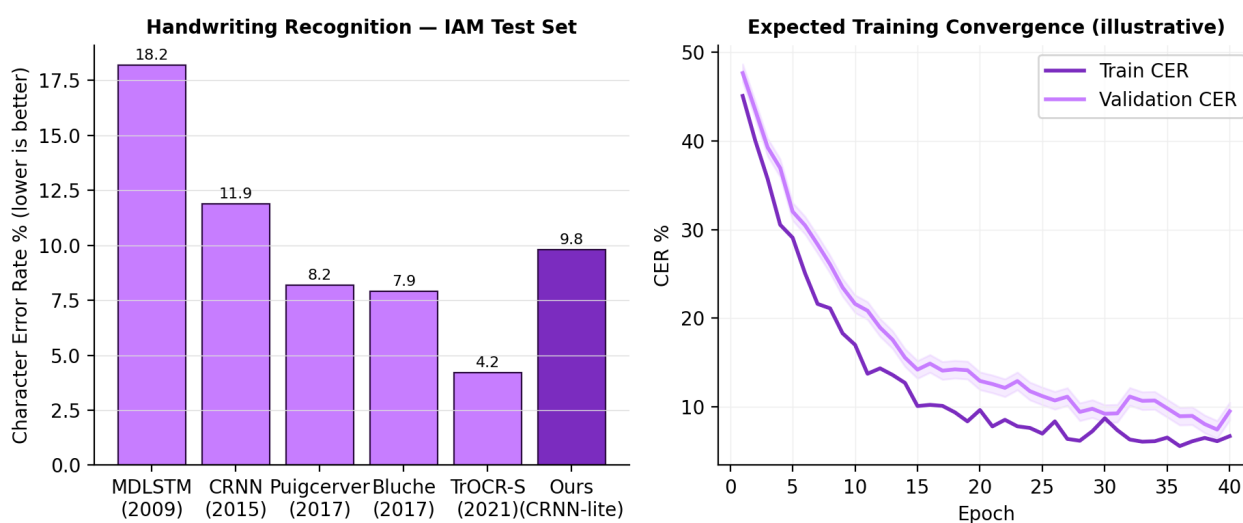


Figure 2.3: (left) Character Error Rate of representative HTR methods on the IAM test set with our expected operating point; (right) an illustrative training/validation CER convergence curve.

12. Innovation / Uniqueness

- **Complete document workflow:** Not just a recogniser — a full pipeline (pre-process $\rightarrow$ segment $\rightarrow$ recognise $\rightarrow$ correct $\rightarrow$ rebuild layout $\rightarrow$ export) wrapped in a usable app.
- **Hybrid accuracy boosting:** Combines a compact CRNN with CTC beam search and a dictionary/edit-distance post-corrector, getting near-Transformer usefulness at a fraction of the compute.
- **Multi-language design:** Pluggable model + dictionary architecture, demonstrated with English and an Urdu/Arabic-script proof-of-concept — relevant to the local context.
- **Low-resource & reproducible:** Lightweight CNN backbone, synthetic-data pre-training and free cloud GPUs make the whole project reproducible on student laptops.
- **Accessibility & preservation focus:** Editable, searchable, screen-reader-friendly output supports visually-impaired users and archival digitisation.
- **Practical export formats:** Direct export to DOCX and searchable PDF (OCR text layer) — features real users actually need, rarely included in coursework projects.

13. Feasibility Analysis

13.1 Technical Feasibility

CRNN+CTC has many open reference implementations; IAM is freely available for research; OpenCV, PyTorch, Tesseract, SymSpell, python-docx and pdf2image are mature. A compact CRNN has only a few million parameters and trains in a few hours on a free Colab T4 GPU, especially when warm-started from synthetic-data pre-training. Inference on a single text line is millisecond-scale on CPU, so the application runs comfortably on student laptops.

13.2 Cost Analysis

Item	Option Chosen	Cost (PKR)
Compute for training	Google Colab / Kaggle free GPU tier	0
Datasets	IAM (free for research) + synthetic data we generate	0
Software & libraries	Python, PyTorch, OpenCV, Tesseract, SymSpell, python-docx (open-source)	0
Development machines	Members' existing laptops	0 (already owned)
Optional Colab Pro / Kaggle add-on	Only if final tuning needs more GPU time	≈ 2 800 / month (optional)
Demo hosting (optional)	Streamlit Community Cloud / Hugging Face Spaces (free)	0
Estimated total	Core project	≈ 0 (PKR 0–3 000 if Colab Pro used)

Table 2.4: Indicative cost analysis — effectively zero-budget.

13.3 Hardware Feasibility & GPU / Resource Limitations

- Training: a free Colab/Kaggle GPU (T4/P100) is sufficient for a compact CRNN; full TrOCR-scale Transformer training is intentionally out of scope.
- Local development/inference: any laptop with 8 GB+ RAM; no GPU strictly required for inference.
- Mitigations for limited resources: synthetic-data pre-training to converge faster, fixed small line-image height to bound memory, gradient clipping and mixed precision, and checkpoint-and-resume to survive Colab session limits.

13.4 Practical Implementation Plan

Weeks 1–4: literature, IAM download, pre-processing/segmentation pipeline, synthetic-data generator, per-character CNN baseline. Weeks 5–8: build and train the CRNN+CTC model on Colab, reach target CER, add beam-search decoding. Weeks 9–12: SymSpell post-correction, multi-language model, layout reconstruction, export module, UI. Weeks 13–16: integration, testing on custom handwriting, evaluation report, documentation and final demo. Roles: pre-processing & segmentation

(member A), model & training (members A & B), post-processing & multi-language (member C), UI/export & documentation (member D), with weekly integration reviews.

14. Hardware and Software Requirements

Requirement	Minimum	Recommended
Processor	Quad-core CPU (Intel i5 / Ryzen 5)	8-core CPU
RAM	8 GB	16 GB
GPU (local, optional)	Integrated GPU (CPU inference fine)	NVIDIA GPU 4 GB+ (e.g. GTX 1650)
GPU (training)	Google Colab / Kaggle free T4	Colab Pro / institutional GPU
Storage	15 GB free	30 GB free (IAM + synthetic data + checkpoints)
OS	Windows 10 / Ubuntu 20.04 / macOS	Ubuntu 22.04
Software	Python 3.10, PyTorch, OpenCV, Tesseract, SymSpell, python-docx, Streamlit, Git	+ CUDA toolkit (if local GPU), VS Code
Peripherals	Scanner or smartphone camera; optional webcam	Flatbed scanner for clean page captures

Table 2.5: Hardware and software requirements.

15. Future Enhancements

- Fine-tune a Transformer recogniser (TrOCR-small) for higher accuracy when more compute is available.
- Full-page layout analysis: multi-column text, tables, forms, marginalia, mixed print/handwriting.
- More languages and scripts (Sindhi, Pashto, Persian) and handwritten-mathematics recognition.
- On-device mobile app (TFLite/ONNX) for instant note capture and digitisation.
- Writer adaptation / few-shot personalisation to a specific user's handwriting.
- Integration with note-taking and document-management systems; batch-folder processing.
- Active-learning loop where user corrections are fed back to improve the model.

16. Timeline / Gantt Chart

The project runs over a $\approx$ 16-week semester in three phases — foundation, modelling & development, and finalisation — with the weekly milestones below.

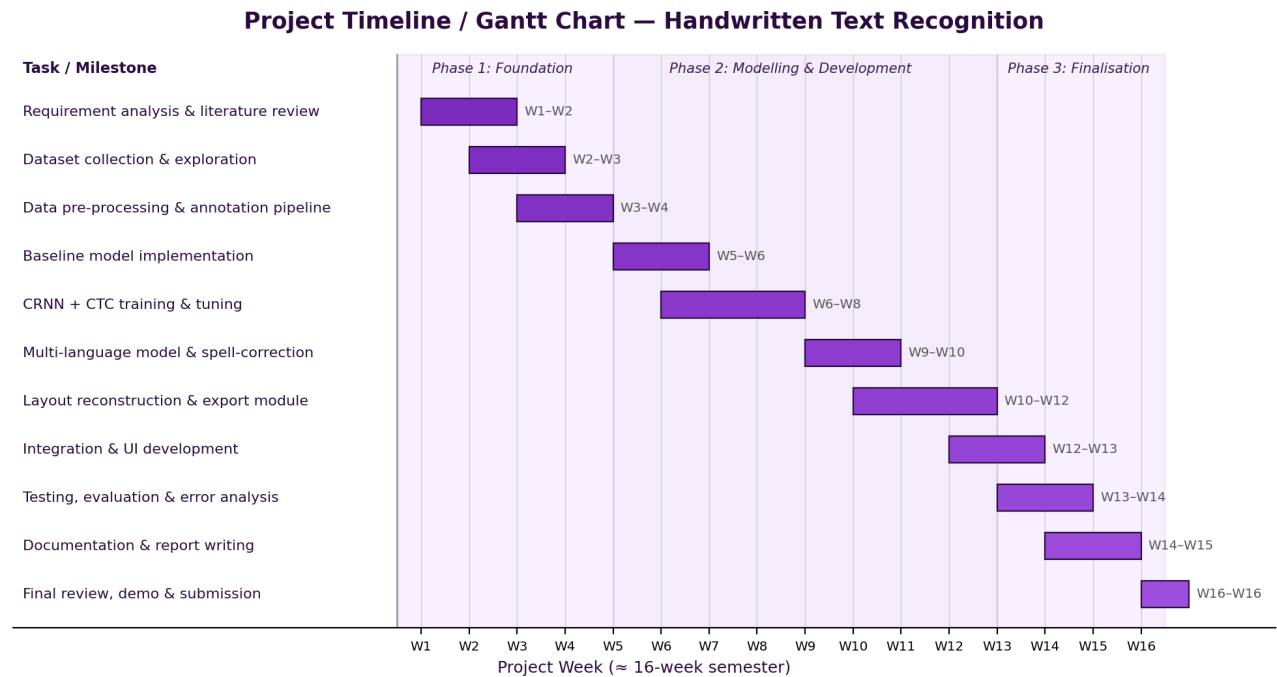


Figure 2.4: Project Gantt chart with weekly milestone breakdown.

Week(s)	Milestone / Deliverable
W1-W2	Requirement analysis, literature review, finalise CRNN architecture and tools
W2-W3	Download & prepare IAM database; exploratory analysis of lines/writers
W3-W4	Pre-processing & line/word segmentation pipeline; synthetic-data generator
W5-W6	Per-character CNN baseline; set up Colab training; build CRNN skeleton
W6-W8	Train CRNN+CTC (pre-train on synthetic, fine-tune on IAM); reach target CER; add beam search
W9-W10	SymSpell post-correction; optional Urdu/Arabic-script model proof-of-concept
W10-W12	Layout reconstruction; export to TXT/DOCX/searchable PDF; build the UI
W12-W13	End-to-end integration; webcam capture mode; performance tuning
W13-W14	Testing on IAM + custom handwriting; CER/WER evaluation and error analysis
W14-W15	Documentation, user guide and final report writing
W16	Final review, demonstration and submission

Table 2.6: Weekly milestone breakdown.

17. Conclusion

This proposal presented an AI-Based Handwritten Text Recognition system that turns images, scans and PDFs of handwriting into editable, searchable text exportable as TXT, DOCX or searchable PDF. Its recognition core is the well-proven CRNN + CTC architecture, kept deliberately compact so it can be pre-trained on synthetic data and fine-tuned on the IAM database using only a free cloud GPU. The project's value lies in delivering the complete document workflow — pre-processing, segmentation, post-correction, layout reconstruction, multi-language support and practical exports — wrapped in a usable application. It is realistic for a four-member team to complete in one semester, builds core competencies in CNNs, sequence modelling and CTC, and yields a tool with genuine utility for students, offices and archives, with a clear path toward Transformer-based and mobile future versions.

18. References

References are listed in IEEE citation style; arXiv / dataset URLs are provided where applicable.

- [1] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298-2304, 2017. [Online]. Available: <https://arxiv.org/abs/1507.05717>
- [2] A. Graves, M. Liwicki, S. Fernández, R. Bertolami, H. Bunke, and J. Schmidhuber, "A novel connectionist system for unconstrained handwriting recognition," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 31, no. 5, pp. 855-868, 2009.
- [3] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proc. Int. Conf. Machine Learning (ICML)*, 2006, pp. 369-376.
- [4] J. Puigcerver, "Are multidimensional recurrent layers really necessary for handwritten text recognition?," in *Proc. IAPR Int. Conf. Document Analysis and Recognition (ICDAR)*, 2017, pp. 67-72.
- [5] T. Bluche and R. Messina, "Gated convolutional recurrent neural networks for multilingual handwriting recognition," in *Proc. IAPR ICDAR*, 2017, pp. 646-651.
- [6] M. Li, T. Lv, J. Chen, L. Cui, Y. Lu, D. Florencio, C. Zhang, Z. Li, and F. Wei, "TrOCR: Transformer-based optical character recognition with pre-trained models," in *Proc. AAAI Conf. Artificial Intelligence*, 2023. [Online]. Available: <https://arxiv.org/abs/2109.10282>
- [7] U.-V. Marti and H. Bunke, "The IAM-database: An English sentence database for offline handwriting recognition," *Int. Journal on Document Analysis and Recognition*, vol. 5, no. 1, pp. 39-46, 2002.
- [8] A. Graves, "Supervised Sequence Labelling with Recurrent Neural Networks," *Studies in Computational Intelligence*, vol. 385, Springer, 2012.
- [9] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. IAPR ICDAR*, 2007, pp. 629-633.
- [10] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *NeurIPS*, 2019, pp. 8024-8035.
- [11] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [12] W. Garbe, "SymSpell: 1 million times faster spelling correction & fuzzy search through Symmetric Delete spelling correction algorithm," 2012. [Online]. Available: <https://github.com/wolfgarbe/SymSpell>

- [13]** Research Group on Computer Vision and Artificial Intelligence, University of Bern, "IAM Handwriting Database." [Online]. Available: <https://fki.tic.heia-fr.ch/databases/iam-handwriting-database>
- [14]** Y. Du et al., "PP-OCR: A practical ultra lightweight OCR system," arXiv:2009.09941, 2020.